WAC
Windows Artefacts Collector

CENTRE D'EXCELLENCE CYBERDEFENSE AEROSPATIALE

Created by the Aerospace Cyber Defence Centre of Excellence (CEC)

Validation report
WAC (Windows Artefacts Collector)

Windows artefact collection software for digital forensic investigation



SYSTEM ARTEFACTS

USER FILES

DIGITAL EVIDENCE

A TOOL IN SUPPORT OF DIGITAL INVESTIGATION

WAC — Windows Artefacts Collector

Validation report

Software version	1.4.0
Document edition	September 2026
Publisher	Aerospace Cyber Defence Centre of Excellence (CEC)
	French Air and Space Force Academy, Air Base 701, Salon-de-Provence

This document lists the checks that establish the quality of WAC: those WAC runs itself during every collection, the test programs that confront its decoders with a reference, and the automatic checks of its outputs. For each, it states what is compared, against what, the reference result and the defects it has already caught. It also states what these checks do not guarantee. How to run each test is described in the *Testing guide*.

Contents

1 The approach **2**

1.1 Why valid JSON proves nothing . . . 2

1.2 Four levels of checks . . . 2

1.3 The development rules that go with them . . . 2

2 The checks built into WAC **4**

2.1 Before any extraction . . . 4

2.2 Raw reading of the NTFS volume . . . 4

2.3 Registry hives . . . 5

2.4 Exhibit integrity: the exhibit store . . . 5

2.5 Authenticity of binaries . . . 6

2.6 Event logs . . . 6

2.7 Decoding the artefacts . . . 6

3 The test programs **8**

3.1 Format harnesses . . . 8

3.2 Conversions: `system_conversions_test` in detail . . . 10

3.3 Robustness harnesses . . . 10

4 The automatic cycle and its checks **11**

4.1 The cycle . . . 11

4.2 The checks of `check-json.py` . . . 11

4.3 Reference result . . . 14

5 Cross-checks on a real collection **16**

6 What these checks guarantee **17**

6.1 Guarantees . . . 17

6.2 Limits . . . 17

CHAPTER 1

The approach

1.1 Why valid JSON proves nothing

WAC produces JSON files meant for a forensic report. Their validity says almost nothing about their accuracy: a field read at the wrong offset, a date interpreted in the wrong time zone, a name decoded in the wrong code page, a wrong decompression all give perfectly valid JSON, and wrong. The most serious defects found in WAC were all of that kind. None showed at run time; each only appeared when a value was confronted with an **independent source**.

WAC's validation therefore rests on one principle: every piece of data that can be cross-checked is, by a means that shares nothing with the code under test.

1.2 Four levels of checks

The checks built into WAC (chapter 2): run during every collection, on the examined machine. They refuse inconsistent data rather than publish it, and attest the integrity of the exhibits taken.

The test programs (chapter 3): sixteen executables that confront each decoder with a reference — the Microsoft function it replaces, a published standard or another implementation — or that prove a parser never reads outside its buffer.

The automatic cycle and its checks (chapter 4): a full collection in a Windows 11 virtual machine, then `check-json.py`, which checks the validity of the outputs and confronts them with one another and with external sources.

Cross-checks on a real collection (chapter 5): what a fresh virtual machine does not contain (old logs, fragmented files, exotic names) is checked on the data of a real machine, processed locally.

1.3 The development rules that go with them

1. **A fix comes with the test that would have caught the defect**, and that test is checked both ways: it passes on the fixed code, and it fails when the defect is reintroduced. A test never seen failing proves nothing.
2. **The build is warning-free**, with `-Wall -Wextra` enabled. The warnings also served as a to-do list: each one marked a field decoded then dropped.

3. **After any change, the new collection is compared with the previous one**, artefact by artefact: an artefact gone from 1,522 entries to 573 lights no warning, only the comparison shows it.
4. **Every input is hostile**, including a file of the examined disk: every read is bounded by the real size of the buffer, never by a length announced in the data.
5. **A field without a value is not emitted**: an empty or null key would read as data found, or as a failed read.

CHAPTER 2

The checks built into WAC

These checks are part of the collection program. They run during every collection, and their result is recorded in `investigation.json`, operation by operation, with its return code and the trace it leaves on the examined system.

2.1 Before any extraction

- Windows version (10 or later) and effective administrator rights (elevated token);
- usable collection location and free space, checked *before* the first extraction: a collection stopped for lack of space would leave an incomplete exhibit store.

2.2 Raw reading of the NTFS volume

Locked files (hives, logs) are read directly from the volume, bypassing the Windows file system. Every structure read is checked before use:

- `FILE` signature of every `$MFT` record and `INDX` signature of every index block, then check of the *fixups* (update sequence array): every 512-byte stride must end with the record's sequence number. A record *torn* by an interrupted write, a mix of two versions, is refused instead of being read as plausible data;
- LZNT1 and WOF decompression bounded by the expected size of each unit: a unit that does not produce the expected number of bytes is an error, not data silently truncated;
- extracted size compared with the size declared by the `$DATA` attribute: their difference, published in the manifest, reveals a truncated extraction a fingerprint alone would not show;
- attributes looked for in the extension records too (`$ATTRIBUTE_LIST`): a directory whose `$INDEX_ROOT` NTFS had moved, the case of thousands of WinSxS components, was reported unreadable (4,569 directories of a Windows 11);
- released index blocks skipped, per the index's `$BITMAP`: they can keep stale entries;
- every attribute read bounded by the attribute's real size, `$STANDARD_INFORMATION` included;
- the source file's dates read in `$STANDARD_INFORMATION` *and* in `$FILE_NAME`: a program forging the former (*timestomping*) leaves the latter untouched, and the gap shows in the manifest;

- volume opened uncached (`FILE_FLAG_NO_BUFFERING`): every read reaches the disk, never an in-memory copy of a sector read earlier; same collection time measured (194 s against 198 s).

2.3 Registry hives

WAC reads the hives with its own reader, linked into the executable, and not with a library of the examined machine.

- base block: `regf` signature, checksum (exclusive OR of the first 127 words), primary and secondary sequence numbers. A hive whose logs have not been replayed is **refused**: read as it is, it would give stale keys without any error;
- replay of the transaction logs (`.LOG1`/`.LOG2`): signature and sequence chaining of every entry; an entry out of the chain is discarded and counted; an undo journal is written before any change, so that the raw copy can be rebuilt;
- every offset read in the hive is checked against the file's real size, and a subkey pointing to one of its ancestors is refused: a walk cannot loop on a forged hive.

2.4 Exhibit integrity: the exhibit store

Every extracted exhibit is placed in an *exhibit store* that is never reopened for writing; all the work is done on a second copy.

- three fingerprints per exhibit (MD5, SHA-1, SHA-256), computed while reading the volume;
- the working copy is checked by its SHA-256 against what was read from the volume, not against a re-read of the exhibit store;
- the manifest identifying the exhibits is sealed by a second file carrying its fingerprint: a retouched manifest is detected;
- a failed extraction is recorded too: an exhibit missing from the manifest would read as never looked for;
- the dates of a file artefact (Prefetch, shortcut, jump list) are the source file's, recorded in the manifest, never those of the working copy, which are the collection's;
- a separate conversion (`--convert`) checks the seal and every exhibit's SHA-256 *before* any read, refuses the store at the first difference, and only accepts exhibit paths that stay inside the store.

A third party can check the exhibit store without WAC, with the scripts `verify-exhibits.ps1` (English messages) and `verifier-consigne.ps1` (French messages), shipped with the user documentation, and their Linux counterparts `verify-exhibits.sh` and `verifier-consigne.sh`, which compute the fingerprints with `sha256sum`, not with WAC. All check the seal, then the fingerprint of every exhibit; an authenticated binary read but not copied ("fingerprint only")

is counted apart, having no file. The manifest coming from the examined machine, an exhibit path that leaves the collection (`..`, absolute path, symbolic link for the `.sh`) is refused without being read: the first versions of the `.ps1` counted as verified a file of the analysis workstation designated by `exhibits\..\..\..`, provided its fingerprint was in the manifest — a defect caught by a counter-test in the test machine. `verify_exhibits_test.sh` checks the two `.sh` on forged collections (23 checks).

2.5 Authenticity of binaries

With the `--binary` option, WAC takes the binaries cited by the artefacts, except those it establishes as signed by Microsoft. That decision rests on a complete verification, done by WAC itself: RSA signature of each security catalog, certificate chain up to a Microsoft root, Authenticode digest of the file found in a verified catalog, or embedded signature. The catalogs read are those of `CatRoot`, of the components (`WinSxS\Catalogs`) and of the servicing packages; the signers accepted, those Microsoft uses for its own code, `.NET` included. The files of a Store application are verified through their package's signature and block map, 64 KiB block by block. A file whose authenticity is not established is collected: doubt leads to collecting, never to discarding. The catalogs, signatures and maps that justified not collecting a file go into the store; every PE carries its build in the manifest (`TimeDateStamp`, `SizeOfImage`), the key of Microsoft's symbol server from which its identical content can be fetched again. With `--binary-all`, everything is collected, and the verdict is recorded for each executable.

2.6 Event logs

Signature of the file, of each chunk and of each record. Every physical chunk of the file is walked, not only those the header declares: a log not properly closed thus yields the records it does not count. An unreadable chunk is reported and skipped; it no longer stops the reading of the file.

2.7 Decoding the artefacts

- every read is bounded by the size of the buffer; an unreasonable number of elements in a list of values (beyond 65,536) is refused rather than walked;
- an impossible date (month 13, hour 25) or one outside the `FILETIME` range gives an absent date, hence not emitted, and never an arbitrary value;
- every local date carries the offset in force *at its date*, according to the daylight saving rules of the examined machine, year by year;
- the SIDs are named from the evidence alone (SAM, well-known SIDs, service SIDs, profiles), never by asking the system or the network: `LookupAccountSidW` queried the domain controller for a SID the machine did not know, a trace of the collection on another machine. The build fails if `WAC.exe` imports a name-resolution function;

- a GUID the reference table does not know is named from the examined machine's hives (machine classes, known folders, each user's classes); unknown everywhere, it is published without a name, never with a placeholder text.

CHAPTER 3

The test programs

Sixteen executables, left out of WAC's build by their `_test.cpp` suffix and built by `build-windows.sh --test`. Each targets one component and confronts it with a judge that shares nothing with it.

3.1 Format harnesses

Program	Judge	Reference result
`sha_test`	FIPS 180-4 vectors, independent implementation	SHA-1 and SHA-256 conform, including lengths 55 to 128 bytes, which exercise the padding
`lznt1_test`	Microsoft's compressor (`RtlCompressBuffer`)	16 compressed units out of 16 conform, 1,048,576 bytes identical
`lzx_test`	Microsoft's compressor (`compact /exe:lzx`)	646 chunks out of 646 matching (three files); 235,707 truncated or damaged inputs without an out-of-bounds read (sanitizers)
`xpress_test`	Microsoft's compressor	137 compressed chunks out of 137 conform, byte for byte
`wevt_test`	files of a real event provider	the whole message production chain checked, substitution marks included
`event_messages_test`	SOFTWARE hive and binaries of the machine	the same chain, step by step, starting from the hive
`evtx_test`	`python-evtx`, independent implementation	record by record; whole chain identical when streamed and in memory (26 MB peak against 1,281 MB)
`offline_registry_test`	Microsoft's `offreg.dll`	18 hives, ten of them from a real machine: 1,273,652 keys and 2,456,139 values identical
`system_conversions_test`	the Windows functions replaced	9,861,570 comparisons, no difference (details below)

Program	Judge	Reference result
`consigne_test`	the exhibit store itself, before and after	exhibit store byte-identical after the working copy was modified; seal conforms; exhibit paths confined to the store
`ntfs_fixup_test`	hand-built records, independent check	sound records restored to the byte, torn records and malformed arrays refused, 20,000 random records against a forbidden page: 44,610 checks, no failure
`mft_test`	`ntfsinfo` (ntfs-3g), an independent NTFS implementation	volume formatted by `mkntfs`: name, parent, size, eight dates and named streams identical for every record; 275 checks, no failure
`usn_journal_test`	the compiler's structures (MinGW's `USN_RECORD`, version 3 from Microsoft)	fields returned as written, numbers equal to the positions in the whole stream, paths through the `$MFT`, damaged records against a forbidden page; 26 checks, no failure. Revealed that the root's sequence was not checked: a parent named by a stale sequence gave a path wrongly asserted
`file_artefacts_test`	each format's published description	PowerShell history, `setupapi.dev.log`, recycle bin `$I` (versions 1 and 2), `hosts`, `Report.wer`, Defender's MPLog: ordinary and hostile cases (truncated `$I`, lying length, impossible date), `Zone.Identifier` streams; 37 checks, no failure
`json_test`	WAC's JSON writer	identical round trip, 26 files of a collection read back and rewritten to the byte, malformed texts refused, 200,000 mutations without an out-of-bounds read (sanitizers)
`raw_hive_test`	Windows's `reg load`	the extracted hive is refused before repair and accepted after: the test fails both ways if it must
`authenticode_test`	`Get-AuthenticodeSignature`	2,233 binaries: 2,126 authenticated by both, 99 collected by both, no file accepted by WAC and rejected by Windows; 462 PowerShell and 11 WSH scripts conform

3.2 Conversions: `system_conversions_test` in detail

WAC depends on five system libraries only (`ADVAPI32`, `KERNEL32`, `msvcrt`, `Secur32`, `WTSAPI32`), needed to query the running system. The conversions it used to ask of other libraries of the examined machine are done by its own code, and this program confronts them with the Windows functions they replace:

- service SIDs, computed offline, compared with `LookupAccountNameW` for the 273 services of the machine;
- GUIDs, SIDs both ways, OLE dates, property names (a table of 1,698 entries generated from the Windows schema);
- FAT dates, on every possible date;
- daylight saving time: offset both ways on the **141 time zones** of Windows, with their rules year by year, at every transition from 1980 to 2035 (hour skipped, hour repeated, change of rule on 1 January);
- reading of the `TimeZoneInformation` key, compared with `GetTimeZoneInformation`;
- time zone rules forged at random: no overflow.

Intended, documented differences: WAC refuses a SID text Windows would truncate, and a non-numeric OLE date for which Windows returns a meaningless time.

3.3 Robustness harnesses

A parser reads at offsets the file itself declares. On a truncated or forged file, a read outside the buffer does not crash on its own: it reads the neighbouring memory and publishes it, in valid JSON. The robustness harnesses make it crash: each input is placed against a forbidden memory page, once at its end, once at its start. A single byte read outside the input stops the test. A watchdog also stops a parser that does not end.

Each file is parsed whole, in its truncations, and in randomly corrupted copies (fixed seed: a failure reproduces identically).

Program	Files of a real machine	Inputs parsed
`lnk_test`	220 shortcuts	546,214
`parsers_test`	48 automatic jump lists	175,585
	23 custom jump lists	79,036
	307 Prefetch files	1,246,491
	4 registry hives	16,207

Result: no read outside the buffer, no endless loop.

CHAPTER 4

The automatic cycle and its checks

4.1 The cycle

`vmtest/run-wac-test.sh` runs, with no interaction at all, a full collection in a Windows 11 virtual machine (real NTFS, real registry):

1. with `--build`, warning-free build of WAC and of the test programs;
2. validation of the raw extraction of a hive: `reg load` must refuse it before repair and accept it after;
3. run of WAC with SYSTEM rights; the exit code is checked, and the trace is searched for any unhandled exception;
4. retrieval of the outputs, of the collection log and of the list of files actually present in the exhibit store;
5. run of `check-json.py`;
6. collection only (`--collect --events --binary`, or `--binary-all`), which must convert nothing; check of the store by `verify-exhibits.ps1`; conversion of a store with one retouched exhibit, which must be refused before any read; conversion, checked by `check-json.py`; second conversion, whose JSON files must be identical to the first's, byte for byte;
7. harmlessness check: two witness files the collection is bound to read keep, before and after it, their three dates and their USN number (`harmlessness.txt`); a control read through Windows changes the access date of one of them, proof that the check can fail. See the *Harmlessness file*.

4.2 The checks of `check-json.py`

The script first checks the JSON validity of every file and the form of the Windows paths (a double escaping shows after parsing). It then confronts values computed by WAC with independent values: another piece of data of the same collection, or an external source. Each check below was written after a real defect, which it caught.

Check	Defect it caught
Dates	

Check	Defect it caught
`X` / `XUtc` pairs, lists included, naming the same instant	double time-zone shift (sessions, shortcuts, installation date); Prefetch run times labelled `+02:00` instead of UTC (1,273 values 2 h off)
offset of every local date / IANA time zone database	every date labelled with the collection day's offset (106 winter dates at `+02:00`); daylight saving dates of the SYSTEM hive read in the wrong layout
no session earlier than the system boot	a two-hour shift on the sessions; later, a boot time 3.5 s late
boot time / Kernel-General 12 event	two independent sources of the same instant: 0.00 s apart
ShimCache date / NTFS date of the file in the manifest	a UTC date treated as local
no event later than the collection	shift or misreading of an event date
FAT dates without a fraction and with even seconds; Amcache text dates without a fraction	every date written with seven digits of fraction, claiming a precision the source does not have (808 dates)
dates of BAM, UserAssist, USBSTOR and Amcache / `taskkill` run by the harness, Prefetch, `Get-PnpDeviceProperty`, PE headers	these four artefacts store UTC, read as local time: all their dates 2 h off; BAM and UserAssist emptied by a regression of the value reading
Decoding	
output key names in English, without abbreviation	`NbVolumes`, `DateLocale`, `executionTime`: an abbreviation, a French word, a lower-case initial
identifier and serial number of a USB key / `Get-PnpDevice`	no key carried its serial number, vendor or product: they are in the key names, not in values
Prefetch hash / suffix of its file name	65 wrong hashes out of 270 (hexadecimal without padding)
Prefetch `\VOLUME{guid}` paths resolved	20,066 paths out of 20,066 unresolved
path and creation date of every exhibit of the system volume in `mft.json` / the exhibit store's manifest (the raw extraction walks the directory indexes down); change journal numbers rising, none after the collection	files with hard links (`System32` and `WinSxS`) appeared under one name only (23,095 exhibits out of 61,242); directories named by their 8.3 alias (`AM3B40~1.945`) when their long name was in an extension record (6,346 exhibits)

Check	Defect it caught
witnesses placed by the cycle (PowerShell history, file sent to the recycle bin, `hosts` entry, startup script with its SHA-256) / `powershell_history.json`, `recycle_bin.json`, `hosts.json`, `startup_folders.json`; file marked as downloaded, its stream read back by Windows / `zone_identifiers.json`, read in the `$MFT`; sections of `setupapi.dev.log` counted by Windows / `setupapi.json`	check added with these artefacts
DestList entries / jump list shortcuts	the first 15 entries of every list lost (573 shortcuts published out of 1,522)
machine name of the events / `OperatingSystem.json`	check of the BinXML decoding, on two unrelated sources
record identifiers unique per log	a stale chunk read again
unresolved `%%nnnn` references in messages	a provider's parameter file not loaded (3,780 messages)
RID found at the end of the rebuilt SID	reading of the SAM database
service states and types, presence of drivers	enumeration filters mistaken for states
no process carries `WAC.exe` among its modules	the tool's modules attributed to the Idle process
plausible `$MFT` references	decoding of the shortcuts' extension blocks
mounts / Windows' reading of the key (`reg query`) and partitions (`Get-Partition`)	6 mounts out of 7 published as "\" (device paths decoded as ANSI)
SID names / `Get-LocalUser`, `Get-LocalGroup`	names asked of the running system, even of the domain controller, and translated into the workstation's language ("Système")
GUIDs unknown to the table / three GUIDs registered by the harness, one per source	"Unmapped GUID" published as a name, among them 18 COM handlers of the VM's tasks, named in its hive
Exhibit store and replay	
seal / real fingerprint of the manifest	a retouched manifest
three fingerprints per exhibit, consistent counts	an unidentified exhibit
files present in the exhibit store / manifest	209 exhibits added after sealing, which every other check let through

Check	Defect it caught
no hive both replayed and repaired	a replay that does not align the sequences
an undo journal for every replay	a raw copy impossible to rebuild
Raw reading, separate collection and conversion	
SHA-256 of files read raw / `Get-FileHash`	safeguard of the reading by batches of contiguous clusters
Prefetch dates / dates given by Windows	0 out of 335 matching: dates of the working copy, the minute of the collection
executables of System32 and SysWOW64 / manifest of a `--collect --binary`	6,446 missing out of 8,540 (`$INDEX_ROOT` in an extension record, names of hard links not declared)
catalogs cited in a verdict / exhibits of the store	170 out of 170 missing (label “catalog” looked for as “catalogue”)
WAC’s Microsoft verdicts / `Get-AuthenticodeSignature`	safeguard: a wrong verdict would leave a binary out of the store
copy of a Store package: untouched, modified, added file	verification through the signed block map
dates of a script forged back to 2001 / `$STANDARD_INFORMATION` and `$FILE_NAME`	safeguard: a forgery must stay visible
retrieval keys / PE headers read by PowerShell	safeguard of the retrievable content
`--binary-all`: copy and verdict of every executable	package check too strict under `--binary-all`
retouched exhibit / refusal of the conversion	safeguard of the check before reading
two conversions of one collection	dates read on the working copy, rebuilt at each conversion

4.3 Reference result

Last cycle, 25 September 2026, on version 1.3.1 (Windows 11, Paris time zone), no failure and no warning:

- **full run** (`--events --binary`, 4 min 33 s): 24 valid JSON files out of 24 produced, USBSTOR included (the harness attaches a virtual USB key on every cycle, so that a recreated test machine knows one too); 62 checks passed, among them 4,090 local dates confronted with the IANA database and 170,344 events decoded from 404 logs; 1,640 exhibits, exactly those of the manifest, seal conforms;
- **collection only** (`--collect --events --binary`, 16 min 39 s): nothing converted; 43,513 executables read, 27,958 authenticated and not copied, 12,700 Windows differential

files, 2,508 collected (676 MB); 6,461 exhibits verified by `verify-exhibits.ps1`; a retouched exhibit refused before any read;

- **conversion** (`--convert`, 43 s): 75 checks passed, among them the 8,539 executables of System32 and SysWOW64 present in the manifest, 637 retrieval keys equal to the PE headers, a timestomped script visible (`$STANDARD_INFORMATION` of 2001, `$FILE_NAME` of the day), the Store package verification; a second conversion gives identical JSON files, byte for byte;
- **`--binary-all`** (previous cycle): 58,509 executables copied, each with its signature verdict.

CHAPTER 5

Cross-checks on a real collection

A fresh virtual machine produces neither fragmentation, nor old jump lists, nor exotic file names, nor damaged logs. Several defects only showed on a real machine, confronted with an independent source:

Cross-check	What it showed
jump lists / `olefile`	the streams are named “1” to “f”, not “01” to “0f”
shortcut target / DestList entry	a wrong code page and an ignored Unicode path
logs / `python-evtx`	a chunk with a wrong signature stopped the reading of the whole file (14 records read out of 270)
EVTX chunks / their CRC32	one-off check: the 1,505 chunks of the 404 logs of the test machine are intact
hives / `offreg.dll`	1,273,652 identical keys
binaries / `Get-AuthenticodeSignature`	no undue authentication
old output / new one	the same files decoded by the current code, compared after normalisation

> **Keep in mind**
> A real collection contains personal data. It is processed locally; a temporary copy in the virtual machine or in a working folder is deleted after use, and nothing of it enters the source repository.

CHAPTER 6

What these checks guarantee

6.1 Guarantees

- **Exhibit integrity**: every exhibit is identified by three fingerprints, the working copy is checked against the reading of the volume, the manifest is sealed, and the exhibit store is never modified (`consigne_test`, checks of the cycle).
- **No writing to the examined system**, apart from the unavoidable traces of a collection on a running system, all recorded in `investigation.json` with the expected trace.
- **Accuracy of the decoders**: where a reference exists — hive reader, decompressions, fingerprints, conversions, time zones, authenticity of binaries — results identical to it on the volumes stated.
- **Robustness**: no shortcut, jump list, Prefetch or hive parser reads outside its buffer or loops, on more than two million truncated or corrupted inputs.
- **Consistency of the outputs**: every possible cross-check between two artefacts, or between an artefact and an external source, is run in every cycle.
- **Independence from the examined machine**: WAC borrows no library from it to interpret the evidence.

6.2 Limits

> **Limit**
> These checks do not prove the absence of defects. They prove that the defects they target are absent, on the data they have seen.

- **Completeness**: an empty artefact cannot be told apart from a machine without traces. Only the comparison between two collections, or with a machine whose content is known, measures it.
- **Data the test machine does not hold**: MTP devices, files spread over several `$MFT` records (`$ATTRIBUTE_LIST`), very large volumes. They are to be validated on real systems.
- **Artefacts without a reference**: when no independent tool reads a format, its accuracy rests on the internal cross-checks and on the public documentation of the format.
- **Time zones**: the confrontation with the IANA database covers the zones whose rules agree with Windows's over the period checked (Western Europe, United States, UTC);

the others are checked against Windows only.

- **Building on Windows**: `build-windows.sh` also runs in the MSYS2 MINGW64 shell, but the checks are made on the Linux build, the one delivered; the MSYS2 build is not run by the test cycle.